

Speculative RAG: Latency-Quality Trade-offs in Multi-Draft Retrieval

Soyoon Park
Columbia University
New York, NY
sp4412@columbia.edu

Rupeet Kaur
Columbia University
New York, NY
rk3408@columbia.edu

Hsuan-Ting Lin
Columbia University
New York, NY
hl3930@columbia.edu

Abstract—Speculative Retrieval-Augmented Generation (Speculative RAG) was proposed as a way to improve both latency and accuracy over standard RAG, but the results in the original paper[1] were demonstrated on multi-GPU hardware. In this work, we recreate the full Speculative RAG pipeline on a single NVIDIA A100 (80GB) and measure how it performs against a standard RAG baseline under identical hardware conditions. We evaluate both systems on the TriviaQA dataset and report latency, accuracy, memory usage, and throughput. Beyond the basic comparison, we also test Speculative RAG under three different inference backends, no optimization, NF4 quantization, and vLLM, to understand how much room there is to improve performance through standard optimization techniques. Our results show that the accuracy and latency benefits reported in the original paper are conditional on two things: running the drafter in parallel across multiple GPUs, and using a fine-tuned drafter model. Without either of these, standard RAG turns out to be both faster and more accurate in our setting. That said, Speculative RAG does show a clear and significant advantage in memory consumption, using far less peak GPU memory than standard RAG, a practical benefit that should not be overlooked. This work contributes a detailed profiling of Speculative RAG under realistic, resource-constrained hardware settings. Our analysis identifies drafter fine-tuning and retrieval acceleration as the two most critical steps needed to recover the performance gains reported in the original paper, and lays the groundwork for future work in that direction.

I. INTRODUCTION

A. Background

Large language models (LLMs) have become quite good at answering questions and generating text that sounds convincing. However, they often struggle when questions require very specific, niche, or recent information that may not have been part of their training data. To help with this, Retrieval-Augmented Generation (RAG) was introduced. The idea behind RAG is straightforward - before answering, the model retrieves relevant documents from an external source and uses them as extra context. This helps reduce factual errors. But RAG comes with a trade-off: pulling in more documents means longer inputs, which slows the model down.

To tackle this latency issue, Wang et al. proposed Speculative RAG [1]. The framework draws inspiration from Speculative Decoding and works by splitting the job between two models. A smaller, specialized model (the drafter) generates candidate answers from different subsets of the retrieved documents. A larger, generalist model (the verifier) then picks

the best one. The authors report that this setup achieves up to 12.97% better accuracy and cuts latency by 50.83% compared to standard RAG.

B. Problem

The results reported by Wang et al. are promising, but they were obtained under specific hardware conditions that may not be accessible to most researchers or practitioners. Running multiple LLMs in parallel, as Speculative RAG does, typically requires significant compute resources. It is unclear whether the same accuracy and latency gains hold up when the system is constrained to a more modest, single-GPU setup. Beyond just reproducing the results, there is also an open question about whether additional optimization techniques can push the performance further, particularly in terms of reducing latency without sacrificing too much accuracy.

C. Scope

This work reproduces the Speculative RAG pipeline under a single-GPU setting and evaluates how it performs compared to standard RAG on the same hardware. We measure accuracy, latency, memory usage, and throughput across both systems. We also apply two optimization techniques, NF4 quantization and vLLM, on top of Speculative RAG and study how each one affects the latency-quality trade-off relative to the unoptimized baseline. The goal is twofold: first, to see how much of the originally claimed benefit survives under tighter hardware constraints, and second, to explore whether these optimizations can make Speculative RAG more practical in resource-limited environments.

II. LITERATURE REVIEW

Retrieval-Augmented Generation (RAG) allows AI models access to given information, so the model can answer the question using external evidence instead of only answering based on memorized knowledge. This also improves the factual accuracy [2]. However, the model needs to read through many retrieved passages in a long prompt, which slows down the inference process. Moreover, the model sometimes ignores important details in the middle of a long document [3].

To solve this problem, some researchers do extra training for models, teaching them to actively filter out unimportant information, or critique the work they generate [4, 5]. But

retraining is expensive, and sometimes the AI model forgets its original information. To avoid expensive training, some researchers use a lightweight evaluator tool to give retrieved documents a relevance score [6]. But this evaluator is too small to have enough reasoning ability to connect multiple pieces of evidence together.

To speed up the generation of answers without expensive retraining, some researchers use a method called speculative decoding. Instead of letting a large model generate every token one by one, they make a smaller model draft possible next tokens, and let the larger model verify them in parallel [7, 8]. Because there is an additional memory cost when running a separate draft model, some researchers make the target model create drafts internally or use extra decoding heads to reduce overhead [9, 10].

Speculative RAG combines these ideas with retrieval. This method does not give all documents to the model at once. It splits retrieved documents into smaller subsets, then generates the answer and rationale drafts from each subset. Next, it uses a verifier to select the best answer [1]. Our implementation follows this idea with limited memory. We use one Mistral-7B-Instruct model for both drafting and verification.

III. METHODOLOGY

A. Models and Data Description

We used the TriviaQA validation split and the DPR Wikipedia passage corpus to evaluate our system. Then, we used facebook/contriever-msmarco to embed the questions. Next, we built a FAISS IndexFlatIP index from the passage embeddings and performed our retrieval over that. Our standard RAG system retrieved the top-10 passages and further combined these passages into a single prompt for mistralai/Mistral-7B-Instruct-v0.1.

We used the same retrieval backend for speculative RAG. The difference is that the retrieved passages were divided into multiple document subsets. Next, we passed each subset to a drafter model, which further generated an answer and rationale. We used the Mistral-7B-Instruct model to serve as both the drafter and verifier because our memory was limited. We did not use the fine-tuned MDrafter or Mixtral-8x7B verifier like the original paper did, so our accuracy was also affected.

B. Optimization Scope and Profiling Methodology

We did not train any new models as part of this reproduction. The drafter model was used zero-shot, meaning it was given a prompt directly without any fine-tuning. The prompt was structured to produce outputs in two parts: a *Rationale* and a *Response*.

For each question, we first retrieved the top relevant documents, then grouped them into clusters using embeddings from the Contriever model. From these clusters, we sampled m subsets, each containing k documents. The drafter then generated a candidate answer for each subset. The verifier scored each

draft using three signals - the drafter’s log probability, a self-consistency score, and a self-reflection score, and selected the highest-scoring answer as the final output.

On the optimization side, we applied two techniques to the Speculative RAG pipeline and compared them against the unoptimized baseline: NF4 quantization, which reduces model memory usage by lowering the precision of model weights, and vLLM, an inference engine designed to speed up generation throughput. For profiling, we used PyTorch Profiler on the unoptimized and NF4 runs, and switched to Nsight Systems for the vLLM runs, as it provides better visibility into GPU-level behavior for that backend.

C. Evaluation Metrics

To measure accuracy, we used the normalized containment metric, which checks whether the correct answer is contained within the model’s response after normalization. This metric is commonly used in benchmarks like TriviaQA and Self-RAG evaluations, making it a reasonable choice for comparison.

For latency, we broke the pipeline down into individual stages: retrieval, subset sampling, drafting, verification, and end-to-end execution, and measured each one separately. We reported both p50 and p95 latency to capture typical and tail-case performance. Beyond latency, we also tracked total pipeline runtime, token counts, generation throughput, and peak GPU memory usage. Together, these metrics give a fuller picture of how each system behaves under our hardware constraints, and make it easier to reason about the practical cost of each configuration.

D. Performance Tuning Methodology

We compared three inference configurations. The first one is a no-optimization HuggingFace baseline, and it uses standard generate() calls. This is inefficient since each draft introduced repeated CPU scheduling and GPU kernel launch overhead. The second one is NF4 4-bit quantization through bitsandbytes. This configuration reduced GPU memory usage a lot. However, it introduced dequantization overhead and did not effectively use Tensor Cores. So, we found that the latency increased and the accuracy dropped. The third one is vLLM with PagedAttention and continuous batching. Under this configuration, the GPU can process drafts more efficiently since m drafter prompts for each question were submitted together. We found that this also reduced the drafting latency and total pipeline runtime, so we think vLLM is the most effective optimization. Since vLLM accelerated drafting, retrieval became the main remaining bottleneck.

IV. EXPERIMENTAL RESULTS

A. Experiment setup

Three configurations were tested:

- **No optimization:** standard HuggingFace generate() with greedy decoding.
- **NF4 quantization:** BitsAndBytesConfig (load_in_4bit = True, bnb_4bit_quant_type='nf4').

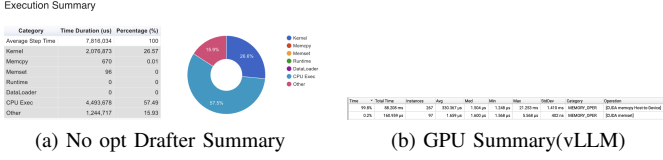


Fig. 1: PyTorch Profiler & Nsys Trace

- **vLLM**: vLLM server with PagedAttention and continuous batching; all m Drafter prompts dispatched as a single async call per question.

Standard RAG and Spec RAG No opt each were run on 1000 TriviaQA validation samples (seed=3) for head-to-head comparison. Speculative RAG without optimization, NF4 quantization and vLLM were run on 100-sample with $m=5$, $k=2$ for comparison. The m -sweep ($m \in \{5, 10, 15, 20\}$, $k=2$) was run under the vLLM backend for hyperparameter analysis.

B. Before and After Optimization

1) *No-optimization*: Without any inference optimization, the Drafter runs in standard FP16 HuggingFace mode. The PyTorch Profiler trace shows an average step time of 7.82ms, with CPU Exec accounting for 57.5% of that wall time (Fig. 1a). GPU utilization is 26.6%, and estimated SM efficiency is 21.0%, which is below the hardware’s potential. The dominant operators are `aten::mm` (25.9% of host self time) and `aten::mul` (16.4%), showing standard FP16 matmul-heavy autoregressive decode. Peak GPU memory allocation is 15797 MB.

Since we used a single GPU, the serial `generate()` calls for each draft create substantial idle time between kernel launches. Because the Drafter processes one draft at a time without continuous batching, the GPU finishes a kernel, returns to Python, dispatches the next token, and repeats. The CPU launch overhead dominates for small-batch autoregressive decoding. End-to-end p50 latency per query is 10681 ms, with the drafting stage alone taking 3937 ms at p50.

2) *After NF4 Quantization*: NF4 quantization reduces the Drafter’s weights from FP16 to 4-bit. A dequantization step is performed on the fly during each linear layer forward pass. Peak GPU memory drops dramatically to 6129 MB, a clear win over the no optimization setup. However, latency actually increases (see Fig. 4a). As evident from the PyTorch Profiler (see Fig. 6), `MatMul4Bit` (bitsandbytes’ quantized kernel) becomes the dominant operator, with 38.2% of host self time, while on-device duration is reported as zero. GPU utilization actually increases to 48.8% and SM efficiency to 45.8% as compared to No opt. However, Tensor Core utilization for `MatMul4Bit` is 0%. Accuracy drops from 54% (No opt) to 48% EM under NF4, consistent with documented 4-bit quality loss on instruction-tuned 7B models.

3) *After vLLM Continuous Batching*: vLLM restructures the drafting step fundamentally. All m draft prompts for a given question are submitted as a single batched request, and

TABLE I: Optimization Comparison (100 Samples, $m=5$, $k=2$)

Metric	No opt	NF4	vLLM
EM (100 samples)	54%	48%	47%
p50 e2e latency (ms)	10681	12881	9131
p50 draft latency (ms)	3937	7662	1932
p50 verify latency (ms)	64.08	98.12	81.24
p50 retrieve latency (ms)	6609	5140	7050
Total pipeline (s)	1518	2007	1157
Peak GPU mem alloc. (MB)	15797	6129	470
Throughput (questions/s)	0.06	0.045	0.082

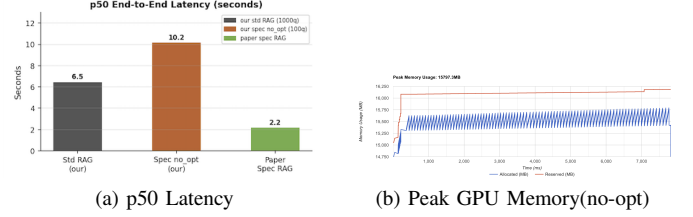


Fig. 2: Performance Comparison & No-opt Profiler Trace

KV cache is managed via PagedAttention and schedules tokens across drafts using continuous batching. The p50 draft latency falls to 1932 ms from the 3937 ms baseline. End-to-end p50 is 9131 ms (see Fig. 4a).

The total pipeline time is 1157s, compared to 1518s for No opt, showing around 25% speedup. Continuous batching eliminates the per-step CPU dispatch overhead seen in the No opt profiler, and PagedAttention removes the KV cache fragmentation that is present in the No opt case (see Fig. 2b for the sawtooth allocation pattern). The GPU operations visible to nsys (see Fig. 1b) show 99.8% of GPU execution time as `cudaMemcpyAsync` Host-to-Device transfers. The 330 μ s average memcopy operations show the scheduling overhead.

Table I summarizes the results across all three backends.

C. Comparative study analysis

1) *Standard RAG (1000 Samples) vs. Speculative RAG No opt (1000 Samples) vs. Paper*: Standard RAG achieves 80% EM on TriviaQA at a p50 end-to-end latency of 6017 ms. Speculative RAG No opt achieves 62% EM at a p50 latency of 9180 ms.

On accuracy, Standard RAG outperforms Speculative RAG due to implementation constraints. The paper reports 74% accuracy on TriviaQA using MVerifier-8x7B with a fine-tuned MDrafter-7B (trained on 40059 instruction-tuning examples). The Drafter here is zero shot: it has never been trained to produce the structured (answer, rationale) format the Verifier’s log probability scoring depends on. The Verifier is also Mistral-7B rather than Mixtral-8x7B, reducing the quality of the final draft selection pass. We did not use Mixtral-8x7B due to memory constraints since nsys traces for even a single sample were of size in gigabytes.

On latency, Standard RAG p50 e2e is 6017 ms vs. Spec RAG’s 9180 ms. Both systems are heavily retrieval-dominated (p50 retrieve $\approx$ 5350 to 5820 ms), but Spec RAG adds extra

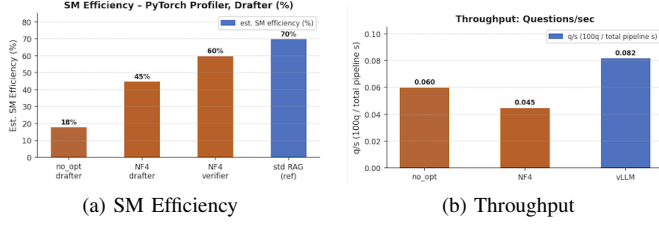


Fig. 3: Before/After Optimization: SM Efficiency, Throughput

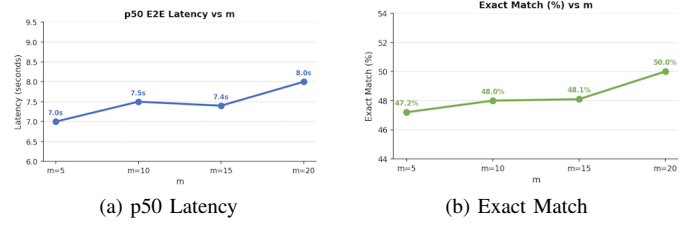


Fig. 5: Hyperparameter m sweep

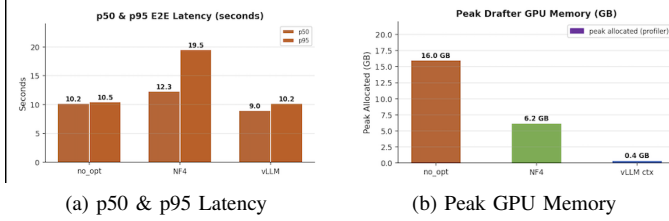


Fig. 4: Before/After Optimization: Latency, Memory

drafting time. Standard RAG generation latency is only 165 ms at p50 (generating a short final answer from a single prompt); Spec RAG’s draft latency is 3733 ms (generating 5 full rationale and answer pairs). The paper shows a latency advantage for Spec RAG since it launches 5 parallel Drafter replicas on 5 GPU endpoints. We could not avail the true parallelism since the experiments were run on a single device without batching. Standard RAG also saturates VRAM at 73066 MB (out of 80 GB) because its long combined prompt fills KV cache. Spec RAG’s per-subset prompts are much shorter, with peak memory at 15797 MB.

2) 100 Samples: No opt vs. NF4 vs. vLLM ($m=5$, $k=2$):

Accuracy: Rationale generation quality degrades slightly due to NF4 quantization, and since the Verifier’s log probability scoring is sensitive to rationale quality, errors compound.

Latency: vLLM delivers the best latency. NF4 is slower than No opt despite being the quantized model since there is a round trip from Python for MatMul4Bit compute with 0 Tensor Core utilization. The result shows the effect of slower drafting negating memory savings. **Memory:** vLLM delivers a clear memory win. **SM Utilization:** There is a large scheduling overhead for no-opt (58% CPU Exec). NF4 improves apparent GPU activity (49%) because dequantization is GPU work, but throughput is lower since it is low-arithmetic-intensity work with 0% Tensor Core usage. vLLM benefits from continuous batching. With drafting accelerated, retrieval now constitutes around 77% of total e2e latency under vLLM (p50 retrieve = 7,050 ms vs. p50 e2e = 9,131 ms). Retrieval has become the clear new bottleneck.

3) Speculative RAG across $m \in \{5, 10, 15, 20\}$ (vLLM, $k=2$): More diverse document subsets give the Verifier more candidates to evaluate, improving the chance of finding the correct answer, hence the EM gain from $m=5$ to $m=20$ (even though marginal). However, it plateaus due to subset redundancy at larger m (see Fig. 5). There is an increase in

latency from $m=5$ to $m=20$ due to the $4\times$ increase in drafts. $m=20$ achieves the best accuracy (52%) at an 8,201 ms p50 e2e. However, the gain is marginal (+5 EM) and it adds cost to latency. So $m=5$ offers the optimal choice without costing latency.

V. DISCUSSION

A. Interpretation of Experimental Results

vLLM outperforms all configurations in speculative RAG. It reduces draft latency by 51%, total pipeline time by 24%, without sacrificing accuracy, owing to continuous batching and PagedAttention. Due to per token dequantization overhead in bitsandbytes, NF4 pays a draft latency penalty. Replacing bitsandbytes with a GPTQ or AWQ backend that supports fused dequant matmul with Tensor Core usage would likely narrow the latency gap. The paper’s fine tuned MDrafter-7B is what leads Speculative RAG to surpass Standard RAG and therefore the discriminativeness of the Verifier’s log probability scoring depends on a tuned drafter to produce good (answer, rationale) pairs. LoRA fine tuning on TriviaQA examples would be expected to improve accuracy.

For Standard RAG, SM utilization meets the proposal’s $\geq 60\%$ target because it runs a single large prompt through one model continuously. For Speculative RAG, short per draft calls, each with smaller batch sizes, leaves the GPU underutilized between dispatches. Retrieval is the large bottleneck. No optimization applied here targets retrieval. FAISS GPU would be expected to close it, however it was not used in our experiments due to memory constraints. The m -sweep confirms vLLM’s batching efficiency does not degrade as m grows. Batching the Verifier scoring calls at high m would allow $m=20$ or beyond to remain competitive on latency.

B. Limitation

Our single-GPU (A100 80GB) hardware setup proved to be the root cause of most experimental decisions that diverged from the Speculative RAG paper. For one, we could not exploit parallelism in the drafter phase as was done in the paper with multiple GPUs; serial execution over m drafting iterations is the largest source of latency disadvantage over measured RAG baseline. For a fair comparison, we could have approximated the post-retrieval latency as (draft latency / m) + verifier latency. Moreover, due to memory limitation, we could not load FAISS-GPU along vLLM, hence we used FAISS-CPU instead with a flat index. Such inefficiency would have

contributed to the retrieval stage being the dominating latency bottleneck (about 77% of total latency in vLLM test case). However, since this was the case for all experiments, it does not affect the comparative latency.

Regarding latency, it is also to be noted that our baseline RAG was run with batch size 32 instead of processing one question at a time as the Speculative RAG does, which gave unfair latency advantage. The paper also implies that their baseline measurement was done by processing questions serially in Section 4.5: “[t]o simulate real-world application scenarios, we process cases individually without batching.” [1]

Regarding accuracy, we observed two main anomalies: not only did Speculative RAG had lower accuracy than standard RAG, but also that baseline RAG was measured to have higher accuracy than was reported in the paper. There are two main explanations for this. First, our drafter model was not fine-tuned as was in the paper, which likely has the biggest impact in the accuracy drop in Speculative RAG. Second, we used a simplified EM metrics which inflates the reported accuracy compared to the EM metrics used in the paper. While this does not explain the relative accuracies between our experiment setups, it explains why our baseline accuracy exceeds that of the paper.

C. Future Work

In aggregate, the highest impact path toward matching the paper’s results is:

- 1) LoRA fine tune the Drafter for structured (answer, rationale) generation. Even a few thousand TriviaQA examples (as opposed to 40k in paper) would show a meaningful improvement in accuracy.
- 2) Use vLLM for both Drafter and Verifier with batched scoring calls.
- 3) Accelerate retrieval via FAISS GPU with a non-flat index.

Quantization adds value only if a larger Verifier (13B or 8x7B) is introduced.

If multiple-GPU setup was possible, we can achieve true parallel drafter execution by deploying m drafter replicas across multiple GPUs, which is expected to bring the latency reduction as claimed by the paper.

VI. CONCLUSION

We re-implemented Speculative RAG under a single-GPU hardware constraint and evaluated it against standard RAG across four key metrics: latency, accuracy, memory usage, and throughput. On top of that, we tested three inference backends: no optimization, NF4 quantization, and vLLM, to understand what practical gains are available when running Speculative RAG in a resource-limited setting. The goal was not just to reproduce the original paper’s numbers, but to stress-test the framework under tighter conditions and understand where its strengths and weaknesses actually lie.

Our experiments produced several concrete findings:

- **vLLM is the most effective backend.** It cut draft latency by 51% and sped up the total pipeline by 24%, all

with minimal impact on accuracy. This makes vLLM a strong default choice for anyone deploying Speculative RAG in practice, especially when inference speed is a priority. The gains come from vLLM’s efficient memory management and batched generation, which allows the drafter to process multiple subsets more quickly than the standard HuggingFace pipeline.

- **NF4 quantization reduces memory but hurts latency.** While it lowers the memory footprint by compressing model weights to 4-bit precision, the additional dequantization overhead during inference adds up and slows the pipeline down. This makes NF4 a useful option only when GPU memory is the hard constraint, and speed can be sacrificed to some degree. It is not a good fit when both memory and latency matter.
- **Standard RAG outperformed Speculative RAG in accuracy- 80% versus 62%.** This was the most significant finding of our study. The gap is largely explained by the drafter being used zero-shot without any fine-tuning. The original paper’s design assumes a drafter that has been specifically trained to produce well-structured rationale-response pairs, which makes the verifier’s scoring mechanism meaningful. Without that, the drafts are noisy and inconsistent, and the verifier ends up selecting from a weak candidate pool. In other words, the accuracy benefit of Speculative RAG is not inherent to the architecture, it is conditional on the quality of the drafter.
- **Retrieval is the biggest latency bottleneck,** not the generation step itself. This was a somewhat surprising result. Intuitively, running large language models for drafting and verification feels like the expensive part. But in practice, the dense retrieval step, searching over a large document index using Contriever embeddings, consumed a disproportionate share of the total pipeline time. This finding shifts the optimization focus away from model inference and toward the retrieval infrastructure, which is often overlooked.
- **Speculative RAG has a strong memory advantage** over standard RAG, using only 15.8 GB of peak GPU memory compared to 73 GB. This difference is substantial. Standard RAG, which feeds all retrieved documents into a single large context window of a big generalist model, is inherently memory-heavy. Speculative RAG, by contrast, distributes the load across smaller document subsets handled by a more compact drafter. Even though the verifier is still a large model, the overall memory profile is much more manageable. This advantage makes Speculative RAG particularly attractive for deployment scenarios where GPU memory is limited.

Taken together, these findings suggest that the benefits reported in the original Speculative RAG paper are not unconditional, they depend heavily on the hardware setup and the quality of the models used. The latency improvements are hard to replicate on a single GPU without the ability to run the drafter and verifier in parallel, which is a key assumption

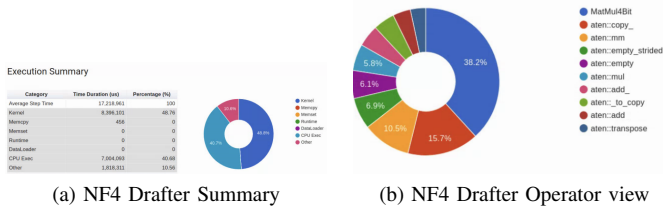


Fig. 6: PyTorch Profiler overview: NF4 drafter

baked into the original design. On a single GPU, the sequential execution of both models adds up, and the retrieval bottleneck further limits how much end-to-end speedup is achievable. Similarly, the accuracy gains depend on having a fine-tuned drafter that can reliably produce structured, high-quality drafts, something that cannot be replicated simply by prompting a general-purpose model zero-shot.

Looking ahead, two directions stand out as the most promising for closing the gap with the original paper’s results. First, fine-tuning the drafter with LoRA would likely have the biggest impact on accuracy. A drafter that has been trained to produce well-reasoned rationale-response pairs would give the verifier a much stronger set of candidates to score, which is where the real accuracy benefit of the framework comes from. Second, switching to FAISS-GPU for retrieval could meaningfully reduce the pipeline’s biggest latency bottleneck. Moving the index search onto the GPU would cut down retrieval time significantly and bring the overall pipeline latency much closer to what the original paper reported. This work establishes a solid instrumentation and profiling foundation, covering per-stage latency breakdown, GPU memory tracking, and throughput measurement, that makes it straightforward to measure the impact of these improvements as next steps.

VII. IMPORTANT LINKS

- **Wandb:** https://wandb.ai/soyoon-columbia-university/hpml_rag
- **GitHub:** <https://github.com/soyoon-cu/speculative-rag.git>

VIII. AI TOOL USE

We used AI assistance as described: Explored and clarified concepts related to Speculative RAG and other ideas described in the paper [1]. Debugged the final report for LaTeX syntax errors after writing the report ourselves. Debugged code for retrieving embeddings instead of directly fetching unembedded passages using indices. Assistance in Linking the GCP bucket for embedded passage retrieval. Assistance in handling failed imports. Explored the use of Nsight Systems for the first time. Tools used: Gemini, ChatGPT. Sections affected: Retriever, Final Report, vLLM. We verified correctness by: Successful imports and results. By submitting this project, we confirm that the analysis, interpretations, and conclusions are our own, and that any AI assistance is fully disclosed above.

REFERENCES

- [1] Zilong Wang et al. “Speculative RAG: Enhancing Retrieval Augmented Generation Through Drafting”. In: *International Conference on Learning Representations (ICLR)*. 2025.
- [2] Patrick Lewis et al. “Retrieval-augmented generation for knowledge-intensive NLP tasks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020.
- [3] Nelson F Liu et al. “Lost in the middle: How language models use long contexts”. In: *Transactions of the Association for Computational Linguistics* (2024).
- [4] Hongyin Luo, Qingxiu Sun, et al. “Sail: Search-augmented instruction learning”. In: *arXiv preprint arXiv:2305.18848* (2023).
- [5] Akari Asai et al. “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection”. In: *International Conference on Learning Representations (ICLR)*. 2024.
- [6] Shi-Qi Yan et al. “Corrective Retrieval Augmented Generation”. In: *arXiv preprint arXiv:2401.15884* (2024).
- [7] Yaniv Leviathan, Matan Kalman, and Yossi Matias. “Fast inference from transformers via speculative decoding”. In: *International Conference on Machine Learning (ICML)*. 2023.
- [8] Charlie Chen et al. “Accelerating large language model decoding with speculative sampling”. In: *arXiv preprint arXiv:2302.01318* (2023).
- [9] Jun Zhang, Jue Wang, et al. “Draft & verify: Lossless large language model acceleration via self-speculative decoding”. In: *arXiv preprint arXiv:2309.08168* (2023).
- [10] Tianle Cai et al. “Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads”. In: *International Conference on Machine Learning (ICML)*. 2024.